LEVEL 3 · DIGITAL DESIGN PROJECT

Status Monitoring Synchronous FIFO

Project description and complete working explanation

Item	Detail
Design module	`status_fifo`
Testbench module	`tb_status_fifo`
Design level	Level 3: FIFO with advanced status monitoring
Modeling style	Synchronous, single clock, counter-based status
Language	Verilog HDL (IEEE 1364 / Verilog-2001)
Development tool	Xilinx Vivado 2025.2.1
Simulator	Vivado Simulator (XSim), behavioral simulation
Developer	Ghulam Yaseen
Date	07 November 2026

1. Project Overview

This project implements a **parameterized synchronous FIFO** (First-In, First-Out buffer) in Verilog-2001 and verifies it with a self-checking testbench. It is the Level 3 design of the series: where a basic FIFO only stores and returns data, this version also **monitors itself**. It reports how many words it holds, raises early-warning flags, and counts every request it had to reject.

Everything runs from a single clock, `clk`, with an active-high synchronous reset, `rst`. All status information is derived from one occupancy counter, which makes the flags simple, consistent with each other, and easy to extend.

1.1 Objectives

- Build a synchronous FIFO with configurable data width, depth and thresholds.
- Generate `full`, `empty`, `almost_full` and `almost_empty` flags from a live occupancy count.
- Track the number of stored words in `count` and handle simultaneous read and write.
- Count rejected writes and rejected reads with saturating counters.
- Use explicit pointer wraparound so that any depth, not only powers of two, is supported.
- Verify reset, filling, full detection, draining, empty detection and both rejection counters with a testbench.

1.2 Deliverables

File	Role
`status_fifo.v`	Design under test (DUT): the FIFO with status monitoring
`tb_status_fifo.v`	Testbench: clock, stimulus and self-checking conditions

> **Key idea**
> The occupancy counter `count` is the single source of truth. `full`, `empty`, `almost_full` and `almost_empty` are plain comparisons of `count`, and the read and write blocks only decide whether an operation is accepted.

2. Theory: Synchronous FIFO Concepts

2.1 What a FIFO Does

A FIFO is a buffer that returns data in exactly the order it was written. It decouples a producer from a consumer: the producer can write in bursts while the consumer reads at its own pace, as long as the

buffer never overflows (write when full) or underflows (read when empty). FIFOs are used in streaming interfaces, communication links, DSP pipelines and any producer-consumer system.

2.2 Circular Buffer and Pointer Wraparound

The storage is an array of `DEPTH` words used as a circular buffer. A **write pointer** `wr_ptr` marks where the next word is stored and a **read pointer** `rd_ptr` marks where the next word is taken. After the last location (`DEPTH-1`) each pointer returns to location 0:

```
0 -> 1 -> 2 -> ... -> DEPTH-1 -> 0
```

This design wraps explicitly with a comparison (`if (ptr == DEPTH-1) ptr <= 0`) instead of relying on natural binary overflow, so it also works for depths that are not a power of two.

2.3 Counter-Based Status

Two common ways exist to tell whether a FIFO is full or empty. One compares the pointers (needing an extra wrap bit). The other keeps an **occupancy counter**, as used here. The counter costs a few flip-flops and an adder, but gives the actual fill level for free and makes threshold flags trivial.

Operation accepted	Effect on count
Write only	count + 1
Read only	count - 1
Write and read together	count unchanged (one word in, one word out)
None	count unchanged

Table 1. Occupancy counter update rules

2.4 Status Flags

Flag	Expression	HIGH at the default settings
`full`	`count == DEPTH`	count = 8
`empty`	`count == 0`	count = 0
`almost_full`	`count >= ALMOST_FULL_LEVEL`	count = 6, 7, 8
`almost_empty`	`count <= ALMOST_EMPTY_LEVEL`	count = 0, 1, 2

Table 2. Status flag definitions

The almost flags are **inclusive**: `almost_full` stays HIGH when the FIFO is full and `almost_empty` is HIGH when it is empty. They are early warnings that a producer or consumer can use to slow down before `full` or `empty` is reached.

2.5 Rejected Operations and Saturating Counters

A request is **rejected** when it cannot be served: a write to a full FIFO or a read from an empty FIFO. The FIFO ignores such a request (no data is stored, no pointer moves) and increments a monitoring counter. The counters are **saturating**: when a counter reaches `{ERROR_WIDTH{1'b1}}` (255 for 8 bits) it stops instead of wrapping to zero, so a large number of errors can never look like none.

3. Specifications

3.1 Parameters

Parameter	Default	Description and constraint
`DATA_WIDTH`	8	Bits in each FIFO word.
`DEPTH`	8	Number of storage locations.
`ADDR_WIDTH`	3	Pointer width; must be large enough to address `DEPTH` locations.
`COUNT_WIDTH`	4	Occupancy counter width; must represent values from 0 to `DEPTH` (8 needs 4 bits).
`ALMOST_FULL_LEVEL`	6	`almost_full` is HIGH when `count` is at or above this level.
`ALMOST_EMPTY_LEVEL`	2	`almost_empty` is HIGH when `count` is at or below this level.
`ERROR_WIDTH`	8	Width of the rejected-write and rejected-read counters.

Table 3. Module parameters

3.2 Port List

Port	Dir	Width	Description
`clk`	in	1	Common clock; all logic acts on the rising edge.
`rst`	in	1	Active-high synchronous reset.
`wr_en`	in	1	Write request.
`data_in`	in	DATA_WIDTH	Data word to write.
`rd_en`	in	1	Read request.
`data_out`	out reg	DATA_WIDTH	Registered output; loaded on an accepted read.
`full`	out	1	FIFO holds DEPTH words.
`empty`	out	1	FIFO holds no words.
`almost_full`	out	1	Occupancy at or above the almost-full level.
`almost_empty`	out	1	Occupancy at or below the almost-empty level.
`count`	out reg	COUNT_WIDTH	Number of valid words stored.
`rejected_writes`	out reg	ERROR_WIDTH	Saturating count of rejected write requests.

Port	Dir	Width	Description
`rejected_reads`	out reg	ERROR_WIDTH	Saturating count of rejected read requests.

Table 4. Port list

3.3 Reset Behavior

When `rst` is HIGH at a clock edge the FIFO returns to its initial state: `wr_ptr = 0`, `rd_ptr = 0`, `count = 0`, `data_out = 0`, `rejected_writes = 0` and `rejected_reads = 0`. The memory array itself is not cleared; its content is irrelevant because the counter prevents a read of a location that was never written.

4. System Architecture

The design is organised as three clocked blocks, one for the write side, one for the read side and one for the counter, around a shared memory array. A small set of continuous assignments decodes the flags from `count` and derives `wr_accept` and `rd_accept`.

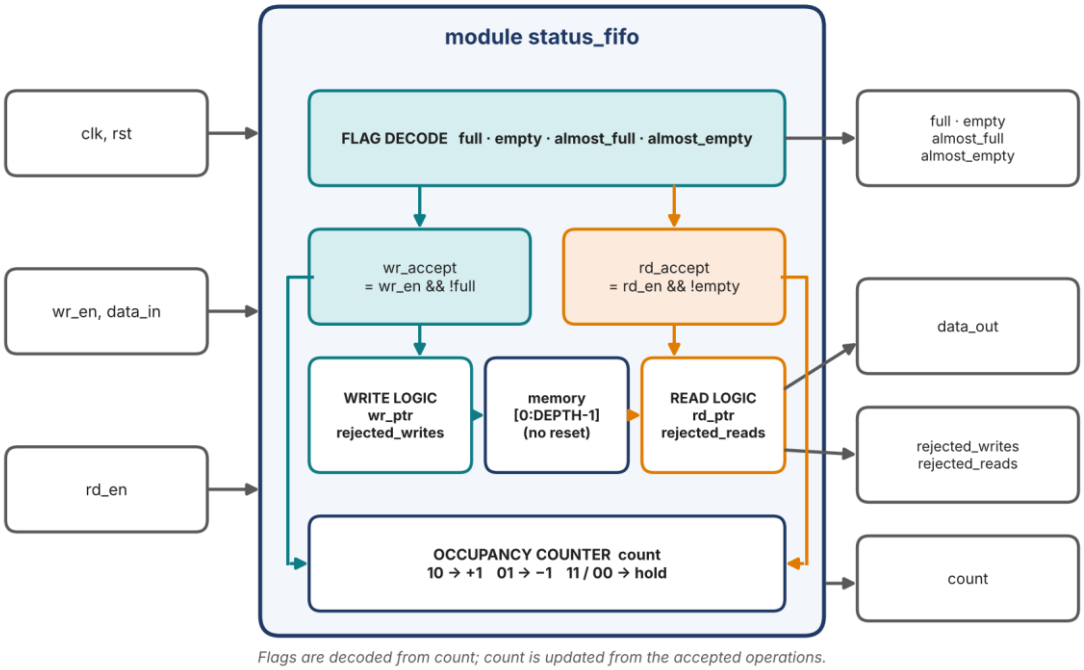


Figure 1. Block diagram of status_fifo

4.1 Internal Signals

Signal	Type	Role
`memory[0:DEPTH-1]`	reg array	Storage for DEPTH words. Written by the write block, read by the read block.
`wr_ptr`	reg	Location of the next accepted write.
`rd_ptr`	reg	Location of the next accepted read.

Signal	Type	Role
`wr_accept`	wire	`wr_en && !full`: the write will really happen.
`rd_accept`	wire	`rd_en && !empty`: the read will really happen.

Table 5. Internal signals

5. Design Walkthrough

5.1 Parameters and Ports

The module header declares seven parameters and the port groups described in Chapter 3. Each port group (clock and reset, write interface, read interface, status flags, monitoring) is separated by a comment banner in the source.

```
module status_fifo #(
    parameter DATA_WIDTH         = 8,
    parameter DEPTH              = 8,
    parameter ADDR_WIDTH         = 3,
    parameter COUNT_WIDTH        = 4,
    parameter ALMOST_FULL_LEVEL  = 6,
    parameter ALMOST_EMPTY_LEVEL = 2,
    parameter ERROR_WIDTH        = 8
)( ... );
```

Listing 1. Module header (abridged)

5.2 Flags and Accepted Operations

```
reg [DATA_WIDTH-1:0] memory [0:DEPTH-1];
reg [ADDR_WIDTH-1:0] wr_ptr;
reg [ADDR_WIDTH-1:0] rd_ptr;

assign full         = (count == DEPTH);
assign empty        = (count == 0);
assign almost_full  = (count >= ALMOST_FULL_LEVEL);
assign almost_empty = (count <= ALMOST_EMPTY_LEVEL);

wire wr_accept = wr_en && !full;
wire rd_accept = rd_en && !empty;
```

Listing 2. Flag generation and accept signals

The flags are combinational functions of `count`. `wr_accept` and `rd_accept` are the gate-keepers: only an accepted operation changes memory, pointers or the counter.

5.3 Write Control Block

```
always @(posedge clk) begin
    if (rst) begin
        wr_ptr <= 0;
        rejected_writes <= 0;
    end
    else begin
        if (wr_en && full && !(rd_en && !empty)) begin
            if (rejected_writes != {ERROR_WIDTH{1'b1}})
                rejected_writes <= rejected_writes + 1'b1;
        end
        if (wr_accept) begin
            memory[wr_ptr] <= data_in;
            if (wr_ptr == DEPTH - 1) wr_ptr <= 0;
            else                     wr_ptr <= wr_ptr + 1'b1;
        end
    end
end
```

Listing 3. Write control block

- **Rejection:** a write request on a full FIFO increments `rejected_writes` unless it saturated. A simultaneous valid read is excluded from this condition.
- **Accepted write:** `data_in` is stored at `memory[wr_ptr]` and the pointer advances, returning to 0 after `DEPTH-1`.

5.4 Read Control Block

```
always @(posedge clk) begin
    if (rst) begin
        rd_ptr <= 0;  data_out <= 0;  rejected_reads <= 0;
    end
    else begin
        if (rd_en && empty && !(wr_en && !full)) begin
            if (rejected_reads != {ERROR_WIDTH{1'b1}})
                rejected_reads <= rejected_reads + 1'b1;
        end
        if (rd_accept) begin
            data_out <= memory[rd_ptr];
            if (rd_ptr == DEPTH - 1) rd_ptr <= 0;
            else                     rd_ptr <= rd_ptr + 1'b1;
        end
    end
end
```

Listing 4. Read control block

The read block mirrors the write block. On an accepted read the word at `memory[rd_ptr]` is loaded into the registered output `data_out`, so the data appears right after the clock edge that accepts the read. When no read is accepted, `data_out` simply holds its previous value.

5.5 Occupancy Counter Block

```
always @(posedge clk) begin
    if (rst) count <= 0;
    else begin
        case ({wr_accept, rd_accept})
            2'b10:   count <= count + 1'b1;   // write only
            2'b01:   count <= count - 1'b1;   // read only
            2'b11:   count <= count;          // both: unchanged
            2'b00:   count <= count;          // idle
            default: count <= count;
        endcase
    end
end
```

Listing 5. Occupancy counter block

The counter looks at the two **accepted** operations, so a rejected request never changes it. Writing and reading in the same cycle leaves the occupancy unchanged, which is the case that lets the FIFO stream continuously at one word per clock.

5.6 Behavior for Every Request Combination

The table below was derived from the RTL equations (and checked with a model of the design). It lists what happens to each combination of `wr_en` and `rd_en` in the three characteristic FIFO states.

State	wr_en	rd_en	Result	count	Counters
Empty	1	0	Write accepted	0 → 1	none
Empty	0	1	Read rejected	0 → 0	rejected_reads + 1
Empty	1	1	Write accepted, read ignored	0 → 1	none
Partly full (4)	1	0	Write accepted	4 → 5	none
Partly full (4)	0	1	Read accepted	4 → 3	none
Partly full (4)	1	1	Both accepted	4 → 4	none
Full	1	0	Write rejected	8 → 8	rejected_writes + 1
Full	0	1	Read accepted	8 → 7	none
Full	1	1	Read accepted, write dropped	8 → 7	none

Table 6. Request combinations and their effect

> **Important behavior to know**
> When the FIFO is **full** and `wr_en` and `rd_en` are both HIGH, the read is served but the write is **not stored** (`wr_accept` is 0 because `full` is 1) and it is also **not counted** as rejected, because the error condition excludes cases with a valid read. The same happens to a read request on an empty FIFO that coincides with a write. See Section 9.2.

6. Testbench Walkthrough

`tb_status_fifo.v` has no ports. It drives the DUT with the default parameters (8-bit data, depth 8, thresholds 6 and 2) and checks four conditions with `$fatal`, which stops the simulation at the first failure.

6.1 Signals and Clock

`clk` starts LOW and toggles every 5 ns (`always #5 clk = ~clk;`), giving a 10 ns period (100 MHz) with rising edges at 5, 15, 25 ns and so on. Inputs are changed on **falling** edges so that they are stable well before the next rising edge. `rst` starts HIGH and is released at 12 ns.

6.2 Test Sequence

Step	Action	Check
1	Hold `rst` HIGH for 12 ns, then release it.	Initial state
2	Set `wr_en = 1` and write the words 0 to 7, one per falling edge.	Eight accepted writes
3	Clear `wr_en`.	`full`, `almost_full` and `count == 8`
4	Raise `wr_en` again with `data_in = EE` for one clock.	Write attempt on a full FIFO
5	Clear `wr_en`.	`rejected_writes != 0`
6	Raise `rd_en` and hold it for eight falling edges.	Eight accepted reads
7	Clear `rd_en` and wait one more edge.	`empty` and `almost_empty`
8	Raise `rd_en` for one clock.	Read attempt on an empty FIFO
9	Clear `rd_en`.	`rejected_reads != 0`
10	Print `LEVEL 3 PASS` and call `$finish`.	Completion

Table 7. Testbench sequence

6.3 The Self-Checks

```
if (!full || !almost_full || count != 8)
    $fatal(1, "Level 3 full flags failed");

if (rejected_writes == 0)
    $fatal(1, "Rejected write was not counted");

if (!empty || !almost_empty)
    $fatal(1, "Level 3 empty flags failed");

if (rejected_reads == 0)
    $fatal(1, "Rejected read was not counted");
```

Listing 6. The four self-checking conditions

Each condition is checked at a falling edge, half a clock after the rising edge that produced the new state, so all registered values have settled.

> **What the testbench does not check**
> The testbench never compares `data_out` with the written values, so **data ordering is not asserted**. The values 0 to 7 do appear in order in the waveform (Section 7.2), but a data error would not stop the simulation. The first stored word is 0, the same as the reset value of `data_out`, so the first read is not even visible as a change.

7. Simulation Flow and Expected Results

> **How these results were obtained**
> No Verilog simulator was available when this report was prepared. The values below come from a **cycle-accurate model of the RTL and testbench timing** (same clock edges, reset, counter, flag and pointer logic). Please reproduce them in Vivado XSim; the expected console output is in Section 7.3.

7.1 Execution Timeline

Time (ns)	Event	count	Flags / counters
0 – 12	`rst` HIGH; first rising edge at 5 ns resets the FIFO	0	empty, almost_empty
12	`rst` released	0	
20	Falling edge: `wr_en = 1`, `data_in = 00`	0	
25	Write word 0	1	`empty` falls
35 – 55	Write words 1 to 3	2 → 4	`almost_empty` falls at 45 (count 3)
65 – 85	Write words 4 to 6	5 → 7	`almost_full` rises at 75 (count 6)
95	Write word 7; `wr_ptr` wraps 7 → 0	8	`full` rises
100	`wr_en` cleared, **check 1 passes**; `wr_en = 1`, `data_in = EE`	8	full, almost_full
105	Write attempt rejected	8	`rejected_writes = 1`
110	`wr_en = 0`, `rd_en = 1`; **check 2 passes**	8	
115	Read word 0 (`data_out = 00`)	7	`full` falls
125 – 175	Reads of words 1 to 6	6 → 1	`almost_full` falls at 135, `almost_empty` rises at 165
185	Read word 7 (`data_out = 07`); `rd_ptr` wraps 7 → 0	0	`empty` rises
190	`rd_en = 0`	0	

Time (ns)	Event	count	Flags / counters
200	**Check 3 passes**; `rd_en = 1`	0	empty, almost_empty
205	Read attempt rejected	0	`rejected_reads = 1`
210	`rd_en = 0`; **check 4 passes**; PASS printed; `$finish`	0	writes = 1, reads = 1

Table 8. Signal values over time

7.2 Waveform

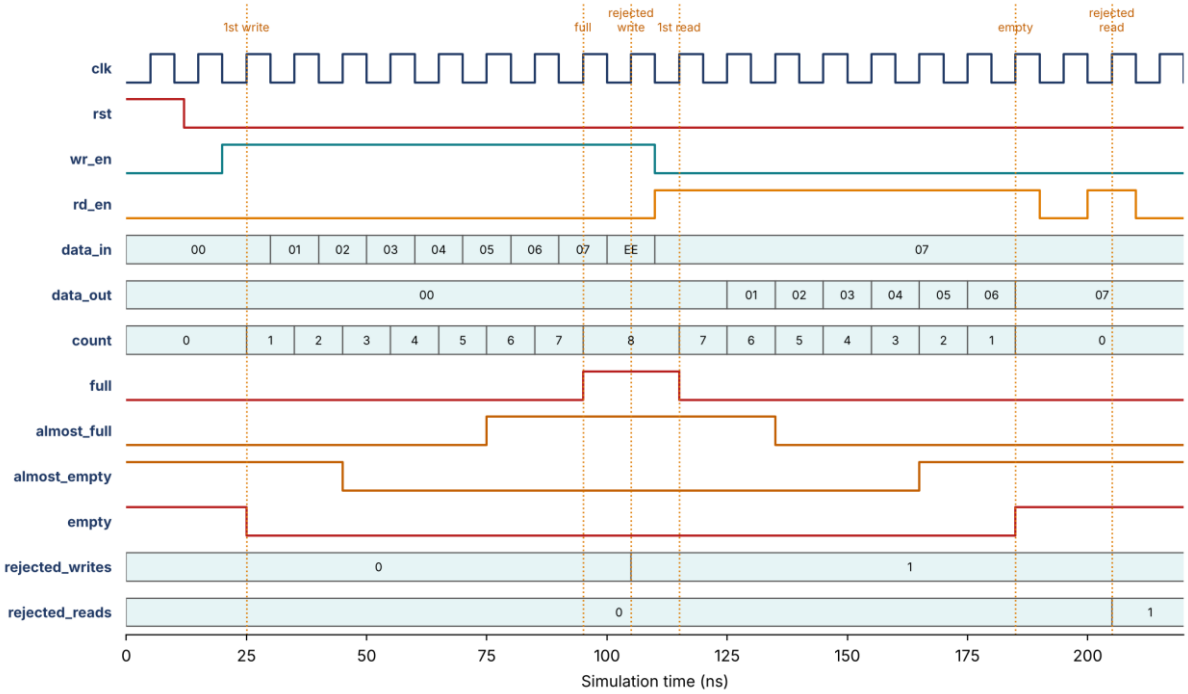


Figure 2. Signal behavior over 220 ns (generated from the cycle-accurate model)

The `count` bus climbs 0 to 8 while the words 00 to 07 are written, stays at 8 during the rejected write at 105 ns, then falls back to 0 as `data_out` steps through 01 to 07. Both rejection counters change exactly once.

7.3 Expected Console Output

```
LEVEL 3 PASS
```

Output 1. Expected message in the Vivado Tcl Console

Vivado also prints a line reporting that `$finish` was called at 210 ns. No other line is printed, because the testbench has no `$monitor` or `$display` calls besides the final message.

7.4 Result Verification

Check	Time (ns)	Required condition	Observed	Status
1. Full flags	100	full = 1, almost_full = 1, count = 8	1, 1, 8	PASS
2. Rejected write	110	rejected_writes ≥ 1	1	PASS
3. Empty flags	200	empty = 1, almost_empty = 1	1, 1	PASS
4. Rejected read	210	rejected_reads ≥ 1	1	PASS

Table 9. Each self-check compared with the expected condition

8. Running in Xilinx Vivado 2025.2.1

Behavioral simulation does not need real hardware, so any device selected during project creation works. The design file is added as a design source and the testbench as a simulation source.

8.1 Vivado GUI Flow

Step	Action	Details
1	Create project	File, Project, New. Choose an RTL project and any device or board.
2	Add design source	Add Sources, then add `status_fifo.v` as a design source.
3	Add simulation source	Add Sources, then add `tb_status_fifo.v` as a simulation source.
4	Check the top module	In Simulation Sources confirm `tb_status_fifo` is the top module.
5	Run simulation	Flow Navigator, Run Simulation, Run Behavioral Simulation. Run to completion with `run all`.
6	Read the result	The message `LEVEL 3 PASS` appears in the Tcl Console. Add signals to the Waveform window to inspect them.

8.2 Command Line

```
xvlog status_fifo.v tb_status_fifo.v
xelab tb_status_fifo -debug typical -s tb_sim
xsim  tb_sim -runall
```

Note on $fatal

`$fatal` is a SystemVerilog system task, while the project is labelled Verilog-2001. If your tool reports an error for it in a `.v` file, compile as SystemVerilog (`xvlog -sv`) or replace the call with `$display` followed by `$finish`.

8.3 What to Look for in the Waveform

- `count` steps up by one per write from 0 to 8 and down by one per read back to 0.
- `almost_empty` falls when `count` reaches 3 and `almost_full` rises at 6.
- `full` is HIGH only while `count` is 8, from 95 ns to 115 ns.
- `rejected_writes` and `rejected_reads` each change once, at 105 ns and 205 ns.
- `data_out` shows 01 to 07 in order after the reads (word 0 equals the reset value 00).

9. Engineering Notes

9.1 Design Properties

Property	Value
Clocking	Single clock, rising edge
Reset	Synchronous, active HIGH
Occupancy tracking	Counter updated from accepted operations
Depth support	Any depth (explicit pointer wraparound)
Read output	Registered; one-clock latency; holds value when idle
Throughput	One write and one read per clock (count unchanged when both occur)
Synthesizable	Yes: memory, counters and comparators map to FPGA RAM/LUTs and flip-flops

9.2 Observations and Recommendations

#	Observation	Recommendation
1	**Silent write drop at full.** With the FIFO full and `wr_en` and `rd_en` both HIGH, the read is served but the write is neither stored nor counted. The in-code comment suggests a simultaneous read is excused, but `wr_accept` still blocks the write.	Either allow the write when a read is accepted (`wr_accept = wr_en && (!full || rd_accept)`), or count the dropped write in `rejected_writes`.
2	**Mirror case at empty.** A read request coinciding with a write on an empty FIFO is ignored and not counted. No data is lost, but the request disappears without being reported.	Count it as a rejected read, or document it as intended.
3	The testbench does not verify `data_out`; word order is only visible in the waveform.	Add a scoreboard: compare each read against the value that was written.
4	Not exercised: simultaneous read/write, intermediate flag levels, counter saturation, more than one pointer wrap, non-power-of-two depth, reset in the middle of operation.	Add directed and randomized tests for these cases.

#	Observation	Recommendation
5	Parameters are not cross-checked. If `COUNT_WIDTH` is too small for `DEPTH` (for example 3 bits for 8), `full` can never become true and the counter wraps.	Add an elaboration-time check, or derive widths from `DEPTH` with `$clog2` (SystemVerilog) or a function.
6	`almost_full` and `almost_empty` are inclusive of `full` and `empty`.	Document this for users of the flags.
7	`$fatal` is SystemVerilog (see Section 8.2).	Compile as SystemVerilog or use `$display` + `$finish`.

9.3 Limitations

- Single clock domain only; crossing clock domains needs an asynchronous FIFO.
- No first-word-fall-through mode and no data-valid output; data is valid one clock after an accepted read.
- Rejected-request counters saturate at the maximum value and are cleared only by reset.
- The memory is not initialised; read data is only meaningful for locations that were written.

9.4 Suggested Improvements

- Resolve observation 1 so that a full FIFO can stream at full rate.
- Add self-checking data comparison and a randomized test with a reference queue.
- Add programmable thresholds as inputs, and sticky overflow/underflow flags.
- Move on to the next level of the series: an asynchronous FIFO with Gray-coded pointers.

Summary

`status_fifo` implements a synchronous FIFO whose behavior is controlled by one occupancy counter. It provides full, empty, almost-full and almost-empty flags, tracks the fill level, wraps its pointers explicitly and counts rejected writes and reads with saturating counters. The testbench fills the FIFO, rejects one write, drains it and rejects one read, and the four checks pass. Before using the design in a larger system, close the simultaneous read/write corner cases described in Section 9.2 and add data checking to the testbench.